

ARM PrimeCell

Smart Card Interface (PL131)

Integration Manual



ARM PrimeCell Smart Card Interface (PL131) Integration Manual

© Copyright ARM Limited 2002. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Smart Card Interface

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	3
4.1	Design Flow Supported	3
4.2	Signal Connectivity	4
4.3	AMBA Bus Connectivity	6
4.4	Non-AMBA Intra-chip Connectivity	6
4.4.1	Interrupts	6
4.4.2	DMA Interface	8
4.5	Primary Input/Output Connectivity	8
4.6	Clocks	8
4.7	Resets	9
4.8	Synchronisation	10
4.9	Scan signals	11
4.10	Endianness	11
4.11	Power Management	11
5	SYNTHESIS	12
5.1	Clock Tree synthesis	12
5.1.1	Setup Scripts	12
5.1.2	Command Script	12
5.1.3	Constraint Scripts	13
5.2	Reset Buffering	13
6	INTEGRATION TESTING	14

6.1	Integration Test strategy	14
6.1.1	AMBA APB signals	14
6.1.2	Non-Amba Intra-Chip signals	14
6.1.3	Primary I/O Pad signals	14
6.1.4	Integration Test Code	14
7	TEST METHODOLOGIES	15
7.1.1	Scan Insertion and ATPG	15
8	FUNCTIONAL AND TIMING VERIFICATION	16
8.1	Functional verification	16
8.1.1	Formal Verification	16
8.1.2	Dynamic simulation	16
8.2	Timing verification	16
8.2.1	Static Timing analysis	16
8.2.2	Dynamic simulation	16
9	CUSTOMISATION	17
9.1.1	FIFO Sizes	17
9.1.2	FIFO Fill levels	17
10	APPENDIX A	18
10.1	PrimeCell Environment Variables	18

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	25 th January, 2002	First release.

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0228	ARM PrimeCell Smart Card Interface (PL131) Technical Reference Manual
3	PL131 DDES 0000	ARM PrimeCell Smart Card Interface (PL131) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	ARM DAI 0005	AMBA Interconnection schemes Application Note 05

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
STA	Static Timing Analysis
SCI	Smart Card Interface

2 SCOPE

This document describes how the ARM PrimeCell SCI may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell SCI is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell SCI block, refer to the ARM PrimeCell Smart Card Interface (PL131) Technical Reference Manual [Ref.2].

The ARM PrimeCell Smart Card Interface (PL131) Design Manual [Ref. 3] contains information about the implementation and design of the ARM PrimeCell SCI block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example microcontroller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TicTalk test vectors for production test.

The AMBA Interconnection schemes Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tristate within an AMBA system [Ref.6].



4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell SCI has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell SCI block.

An example ASIC design flow is described below:

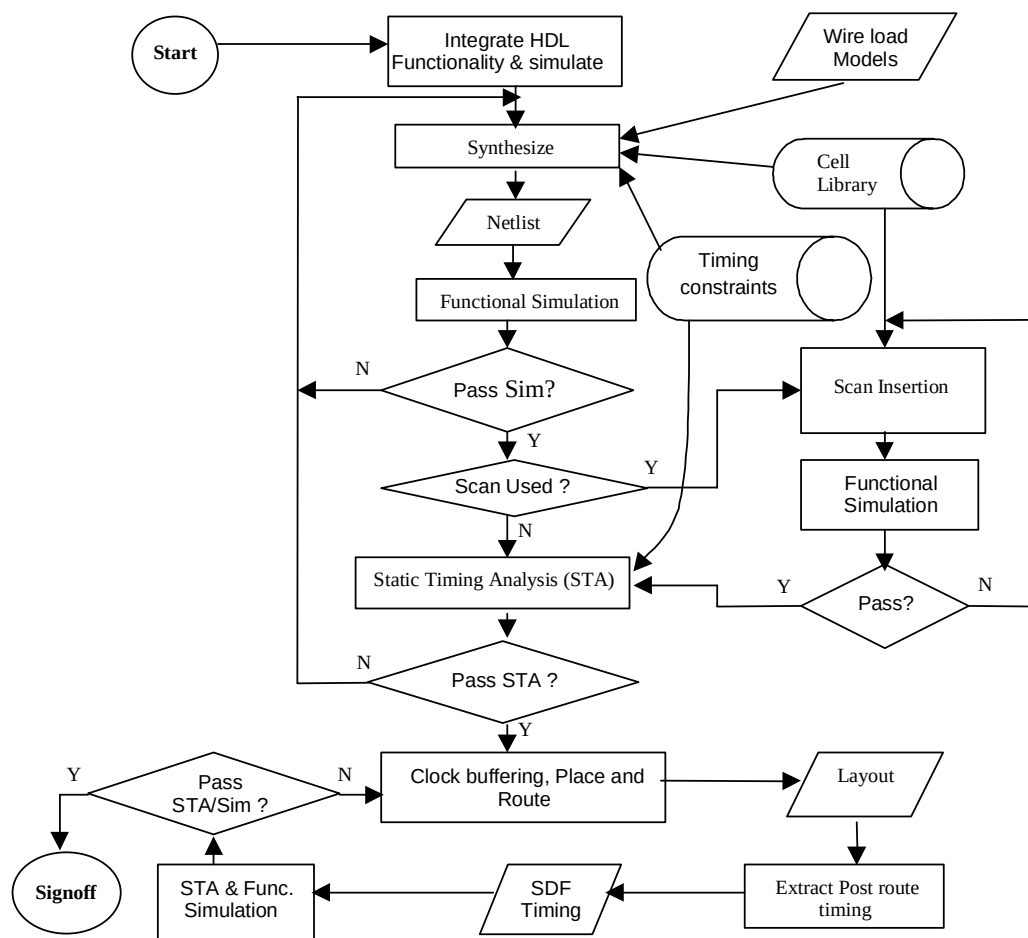


Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell SCI block. For further information refer to ARM PrimeCell Smart Card Interface (PL131) Technical Reference Manual [Ref.2].

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB clock, used to time all bus transfers.
PRESETn	IN	Reset Controller	Bus reset input signal (active low).
PADDR [11:2]	IN	APB Bridge	Subset of APB address bus.
PWDATA[15:0]	IN	APB Bridge	Subset of unidirectional APB write data bus. Data is written into the block via this bus..
PRDATA[15:0]	OUT	APB Bridge	Subset of the unidirectional APB read data bus. Data is read out of the block via this bus.
PENABLE	IN	APB Bridge	Enable signal, asserted HIGH for one PCLK period to enable a bus transfer.
PWRITE	IN	APB Bridge	APB transfer direction signal, indicates a write into the block when HIGH, a read from the block when LOW.
PSEL	IN	APB Bridge	When HIGH, this signal indicates that the block has been selected by the APB Bridge.

Table 4.2-1: APB Signals

Signal Name	Type	Source / Destination	Description
SCICLK	IN	Clock Generator	SCI Reference clock
nSCIRST	IN	Reset Controller	Reset signal to the SCICLK domain (active low).
SCICARDININTR	OUT	Interrupt Controller	SCI Card In interrupt.
SCICARDOUTINTR	OUT	Interrupt Controller	SCI Card Out interrupt.
SCICARDUPINTR	OUT	Interrupt Controller	SCI Card Powered Up interrupt.
SCICARDDNINTR	OUT	Interrupt Controller	SCI Card Powered Down interrupt.
SCITXERRINTR	OUT	Interrupt Controller	SCI Character transmission error interrupt.
SCIATRSTOUTINTR	OUT	Interrupt Controller	SCI ATR start timeout interrupt.
SCIATRDOUTINTR	OUT	Interrupt Controller	SCI ATR duration timeout interrupt.
SCIBLKOUTINTR	OUT	Interrupt Controller	SCI Block timeout interrupt between blocks.
SCICHTOUTINTR	OUT	Interrupt Controller	SCI Character timeout interrupt between characters.
SCIRTOOUTINTR	OUT	Interrupt Controller	SCI Receive FIFO Read timeout interrupt.
SCIRXTIDEINTR	OUT	Interrupt Controller	SCI Receive FIFO tide mark reached interrupt.
SCITXTIDEINTR	OUT	Interrupt Controller	SCI Transmit FIFO tide mark reached interrupt.
SCIRORINTR	OUT	Interrupt Controller	SCI Receive Over Run interrupt
SCICLKSTPINTR	OUT	Interrupt Controller	SCI Clock Stop interrupt.
SCICLKACTINTR	OUT	Interrupt Controller	SCI Clock Active interrupt.
SCIINTR	OUT	Interrupt Controller	SCI interrupt. Combined interrupt generated as a logical

			OR function of the fifteen sources.
SCITXDMASREQ	OUT	DMA Controller	SCI Transmit DMA Single request.
SCIRXDMASREQ	OUT	DMA Controller	SCI Receive DMA Single request.
SCITXDMABREQ	OUT	DMA Controller	SCI Transmit DMA Single request.
SCITXDMABREQ	OUT	DMA Controller	SCI Transmit DMA Single request.
SCITXDMACLR	IN	DMA Controller	SCI Transmit DMA request clear.
SCIRXDMACLR	IN	DMA Controller	SCI Receive DMA request clear.
SCANENABLE	IN	Test Controller	SCI Scan enable signal for both clock domains.
SCANINPCLK	IN	Test Controller	SCI input scan signal for the PCLK domain.
SCANINSCICLK	IN	Test Controller	SCI input scan signal for the SCICLK domain.
SCANOUTPCLK	OUT	Test Controller	SCI input scan signal for the PCLK domain.
SCANOUTSCICLK	OUT	Test Controller	SCI input scan signal for the SCICLK domain.

Table 4.2-2: On-Chip Signals

Signal Name	Type	Source / Destination	Description
SCICLKIN	IN	PAD	SCI clock input.
SCIDATAIN	IN	PAD	SCI serial data input.
nSCICLKOUTEN	OUT	PAD	Tristate output buffer control for Clock Pad (active low).
SCICLKOUT	OUT	PAD	Clock output.
nSCIDATAOUTEN	OUT	PAD	Data output enable, typically drives an open-drain configuration (active low).
nSCICLKEN	OUT	PAD	Tristate control for external off-chip Clock buffer (active low).
nSCIDATAEN	OUT	PAD	Tristate control for external off-chip Data buffer (active low).
SCIVCCEN	OUT	PAD	Supply voltage control.
nSCICARDRST	OUT	PAD	Reset to Card (active low).
SCIFCB	OUT	PAD	Function code bit, used in conjunction with nSCICARDRST.
SCIDETECT	IN	PAD	Card detect signal from card interface device.
SCIDEACREQ	IN	PMU	Card deactivation request signal from the PMU.
SCIDEACACK	OUT	PMU	Card deactivation acknowledgement to the PMU.

Table 4.2-3: Signals to I/O pad cells

4.3 AMBA Bus Connectivity

The SCI block is designed to be integrated into an AMBA system via the APB (Advanced Peripheral Bus.) It is designed to work in systems where an AMBA bus master can read from or write to the SCI through the APB bridge (from ASB or AHB) in response to interrupt signals.

An example APB Bridge is available within the ARM Micropack product. The decode logic within the APB Bridge will need to be altered to take into account the address location of the SCI and any other APB peripheral blocks in the memory map. The base address of the SCI needs to be defined on a system specific basis and the bridge decode logic altered accordingly. The APB Bridge appears as a slave block on the ASB with a decode signal, which must be altered in the HDL code of the AMBA decoder block, to select the correct range for the APB blocks (including the SCI).

The SCI block interfaces with the AMBA APB interface using separate uni-directional read (output) and write (input) data buses PRDATA and PWDATA. For further information refer to the APB [Rev E] Specification [Ref.1].

The data bus may be interfaced at the APB Bridge either using multiplexers or logical OR'ing of data buses from other APB peripheral blocks. In common with other APB peripheral blocks, the SCI block forces all read data bus lines low when there are no APB read transfers in progress.

However, it is also possible to interface the SCI block with a 3-state bi-directional data bus. A wrapper needs to be added around the SCI block connecting the read and write data buses to bi-directional drivers. Control of the 3-state drivers can be done using the APB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and achieving high coverage fault grading.

4.4 Non-AMBA Intra-chip Connectivity

The Non-AMBA intra-chips signals consist of two major sets; the interrupt signals which terminate on an interrupt controller and those associated with the request and clear signals of a DMA interface.

4.4.1 Interrupts

The SCI block provides flexibility in choice of an interrupt handling strategy by providing both individual and combined interrupts. Fifteen interrupting conditions (SCICARDININTR, SCICARDOUTINTR, SCICARDUPINTR, SCCARDDNINTR, SCITXERRINTR, SCIATRSTOUTINTR, SCIATRDOUTINTR, SCICHTOUTINTR, SCIBLKOUTINTR, SCIRTOOUTINTR, SCIRXTIDEINTR, SCITXTIDEINTR, SCIRORINTR, SCICLKSTPINTR and SCICLKACTINTR) are indicated by separate interrupt lines. These conditions are also logically OR'ed together to provide a single combined interrupt, SCIINTR.

The interrupt architecture could use one of the two schemes below:

- 1 Each peripheral provides a single combined interrupt source to the system interrupt controller, which is the OR function of the internal masked sources. The system interrupt controller may then provide another level of masking on a per peripheral basis.
- 2 Each peripheral passes all of its individual sources to the system interrupt controller which would also provide the mask registers for each individual source. In this way a global interrupt service routine would be able to read the entire set of sources from one wide register in the system interrupt controller.

Figure 4.4-1 illustrates how a typical peripheral block outputs both the individual masked interrupts as well as a combined output.

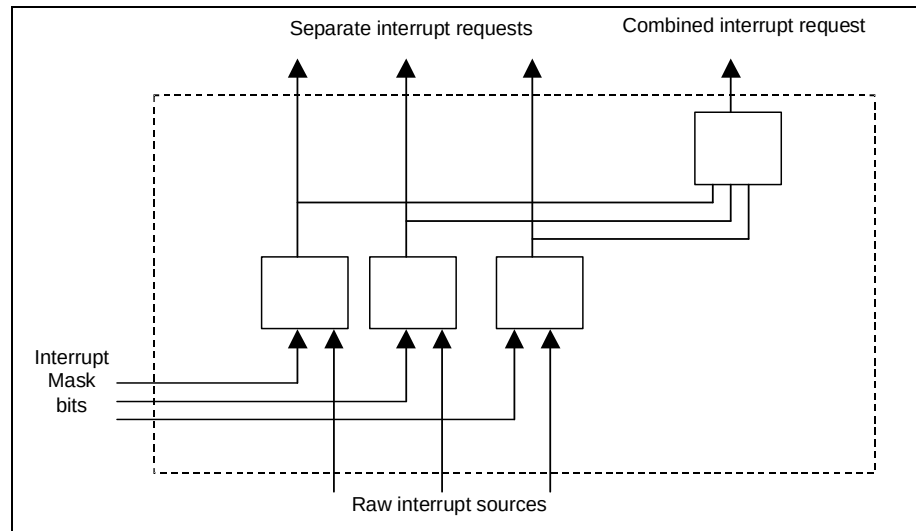


Figure 4.4-1: Interrupt structure

The former system has the advantage of allowing modular device drivers which always know where to find the interrupt source control register bits. The latter system has an advantage where a more integrated interrupt service routine is required for the system and it would be especially attractive where the time to read from the peripheral registers is significant compared to the CPU clock speed in a real time system.

The SCI block has been designed to support system architectures which use either type of interrupt controller since the overhead for doing so is small.

Software may be implemented with either separate interrupt service routines for each interrupt or with a single interrupt service routine which polls each of the two interrupt status bits to determine the interrupt source. Each of these interrupt sources can be masked individually within the SCI itself, thus complementing the mask bits provided in the interrupt controller. The ARM core has two interrupts – nIRQ and nFIQ. Normally a low priority peripheral block such as the SCI would route the interrupt source to the nIRQ input of the ARM core.

4.4.2 DMA Interface

A dedicated 6-wire DMA interface has been built into the SCI to connect to PrimeCell DMA controllers. [Please refer to the ARM PrimeCell SCI (PL131) Technical Reference Manual [Ref.2] for a description of this interface.]

If the SCI is being integrated into a system where the dedicated DMA interface is not used, the SCITXDMACLR and SCIRXDMACLR lines should be connected to a logic '0'.

4.5 Primary Input/Output Connectivity

The clock and data pins of the SCI that are connected to the card contacts can be configured such they behave as open drain or as true push-pull buffered outputs. For details of these configurations, please refer to the SCI Technical Reference Manual [Ref. 2], Section 6 PrimeCell clock and data driver configurations.

The remaining signals SCIFCB and nSCICARDRST drive the card contacts directly.

The card detection signal is provided by the card reader, whilst a power management unit can supply the deactivation request signal (SCIDEACREQ) and monitor the status through the acknowledge signal (SCIDEACACK). Deactivation powers down the card in a controlled manner; it does not deactivate the SCI.

4.6 Clocks

The SCI HDL contains two clock sources at the top level – PCLK and SCICLK. There are two possible clocking options, depending on the required configuration:

- a) Clock both sources with the AMBA system clock, PCLK. This can be done if the Bus Clock frequency is suitable for creation of the required SCI bit duration rates using the dividers implemented in the SCI. Use of a single clock source allows easier static timing analysis and the use of a single scan chain (if scan insertion is being used). This is the recommended approach for this design.

The APB Bridge could also be designed so that PCLK is an integer sub-multiple of PCLK if a lower frequency bus clock is required in order to reduce power consumption.

- b) Clock PCLK and SCICLK from separate sources. This is required when the system bus clock frequency is not suitable for generating the required bit clock rates. If separate asynchronous clocks are used, each clock domain can be individually statically analysed and the simplest strategy is to use two separate scan chains with no mixing of clocks or edges. It is also possible to implement a single scan chain mixing different clocks and scan tool vendor documentation should be consulted for further details.

It should be noted that the SCI design assumes that both the clock sources are free-running during normal device operation. While gating the clocks is not recommended, it might have to be employed in very power critical applications. It is imperative that the clocks are shut off only after the SCI is completely idle. Accesses to the SCI are not guaranteed to complete correctly if either clock is not free-running. Clocks should only be turned off when the receive and transmit operations have completed.

Additionally, if clock gating is required, it should be implemented in a separate block to ensure ease of integration of the SCI block in a typical design flow.

For successful clock tree insertion, there must be a clear path from the PCLK source to SCICLK so that the clock trees PCLK and SCICLK will be balanced to each other. Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default SCI synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools which understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.7 Resets

The SCI block has one reset pin for each of its internal clock domains.

The system bus reset signal PRESETn, resets the PCLK domain registers. PRESETn can be asserted asynchronously, but must be synchronously de-asserted with respect to PCLK. A separate reset pin nSCIRST, is used to reset the SCICLK domain registers.

This signal needs to be provided by a separate Reset Controller, and it must be de-asserted synchronously to SCICLK. T

he resets can be asserted asynchronously even if no clocks are present to ensure that signals from bi-directional I/O cells have defined values during reset, thus avoiding static power consumption problems.

While re-synchronisation of PRESETn to SCICLK could have been done within the SCI block itself, the function was moved out to eliminate internally generated resets.

Figure 4.7-1 below illustrates a method of generating nSCIRST from PRESETn, such that it will be asserted asynchronously and de-asserted synchronous to SCICLK. The signal nSCIRST de-asserts two SCICLK cycles after PRESETn is de-asserted to prevent metastability problems.

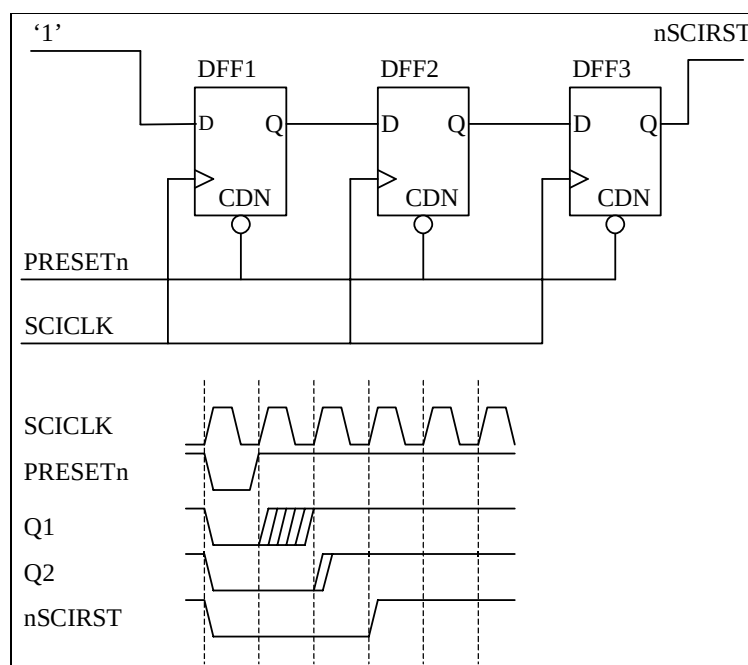


Figure 4.7-1 A method of generating nSCIRST

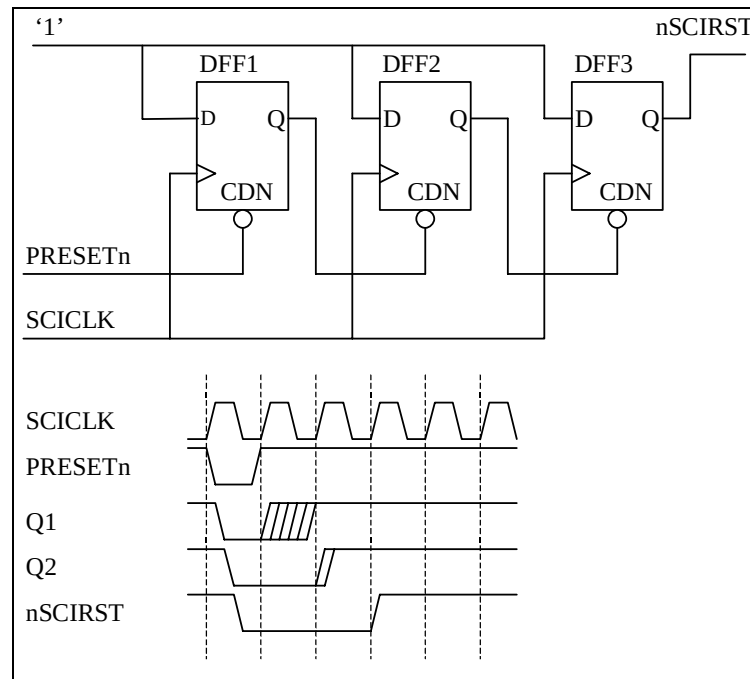


Figure 4.7-2 An alternative method of generating nSCIRST

The method of generating nSCIRST in Figure 4.7-1 could experience simulation problems depending on the cell library modelling. The Q output of DFF3 should not be metastable when setup/hold violation occurs due to de-assertion of PRESETn, since both D and Q are both '0', however the DFF model may assign 'X'.

An alternative method of generating nSCIRST is therefore illustrated in figure 4.7-2 which achieves the same functionality. This method does use internally generated resets, which can present problems in Static Timing Analysis, scan insertion and ATPG, but they are limited to the reset controller at the top-level of the design hierarchy.

This peripheral contains a small amount of logic in the reset net to implement a test reset signal in order to increase fault coverage when production testing with TicTalk test vectors. The logic in the reset nets is located within a separate hierarchical sub-block SciTest.

4.8 Synchronisation

Single bit signals between the two clock domains are synchronised by two series flip-flops inferred during synthesis from standard HDL templates. Multiple bit vector signals use an update mechanism with synchronised single bit control signals to avoid meta-stability problems.

Some technology libraries offer meta-stability hardened D flip-flops specifically for implementing synchronisers. All the synchronisers in the SCI block have been isolated to the sub-blocks SciSyncToPCLK and SciSyncToSCICLK. This allows the synthesis scripts to be modified so that these blocks could be synthesized separately to use the synchroniser flip-flop cells using the Synopsys Design Compiler set_prefer command. Further synthesis of these sub-blocks can then be prevented using the set_dont_touch command. The rest of the SCI design is then synthesized after placing a set_dont_use command on the synchroniser cells. This method also offers the opportunity to simulate the gate-level netlist with asynchronous clocks PCLK and SCICLK and eliminate the

expected setup and hold violations on the synchronisers from the simulation transcript, since the synchronisers cells can be modified to remove 'X' propagation.

4.9 Scan signals

The SCI contains two clock domains that are assumed to be asynchronous.

To support this, scan inputs (SCANINSCICLK and SCANINPCLK) and scan output signals (SCANOUTSCICLK and SCANOUTPCLK) for the SCICLK domain and the PCLK domain scan chains respectively are present at the top level of the SCI.

A common SCANENABLE signal is used to switch the SCI into scan test mode. If the clocks in the final design are synchronous, then it is possible to use only a single scan chain.

4.10 Endianness

Although the SCI is designed for a system using little-endian memory organisation, it is equally possible to integrate the block in a system using big-endian memory organisation.

Since the SCI has a 16-bit wide APB data interface it can be connected in a little endian system to bits 15:0 of the data bus and use half word-wide accesses for maximum performance.

For a big-endian system, to maximise the system performance, the SCI block should be connected to bits 31:15 of the data bus. All half-word accesses to the block will then appear at an address offset of zero. The zero address offset then allows simple generation of the register address in ARM code using a 16-bit shifted constant. A half-word load is preferred to a word load since software would need to mask out the unused bits. However, this method of connection in a big-endian system requires a script to transpose the bytes of the 32-bit wide TicTalk test vectors so that they will be applied correctly to the SCI block now connected to bits 31:15 of the data bus, unless scan testing is used instead of AMBA TIC testing.

Alternatively, in a big-endian system, the SCI block could be connected to bits 15:0 of the data bus and only word accesses used for this block. Since the unused bits will be undefined, software must now mask them out making the code less efficient. The TicTalk test vectors no longer need to have the byte order transposed however.

4.11 Power Management

It is recommended that if stopping of the clocks is required, then this be done in a controlled manner and only when the SCI is truly idle. Data within the SCI is retained when the clock is stopped.

5 SYNTHESIS

5.1 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default SCI synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that a separate tool, such as place & route tools that understand placement, further on in the backend process will perform clock buffering. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

The SCI block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script - Sci.cmd - synthesises the SCI to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the Sci_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

The synthesis scripts consist of the following three main categories:

5.1.1 Setup Scripts

Scripts that are common to several designs are located in the \$GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The global.scr script may need modifying to define:

- target area goals
- maximum transition costs

The library_avanti_cb25_v2.1.scr script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also good practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the library_avanti_cb25_v2.1.scr file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The \$LIB_SYNOP directory must contain the files, or links to the target cell library files, in Synopsys format.

5.1.2 Command Script

The master compile command script Sci.cmd in the synopsys/scripts directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The Sci.cmd script includes setup and

constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script `Sci_scan.cmd` in the `synopsys/scripts` directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables, which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- \$PERIPH is assigned to 'topname' to define the name of the top level module in the design.
- \$HDL_SOURCE is assigned to 'hdl' to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- \$HDL_NETL is assigned to 'hdl_out' to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

5.1.3 Constraint Scripts

- a) Operating conditions and signal load/drive values should be set for the chosen technology library in the `library_avanti_cb25_v2.1.scr` file.
- b) Timing constraints for APB bus signals are set via the `apb_params.scr` and the `apb_slave.scr` file. As delivered, the timings are set for a 100 MHz clock. These timings should be adjusted according to the target frequency required. The `apb_params.scr` file has been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- the minus uncertainty value is set at 5% of the clock period.
- the plus uncertainty value is set at 40% of the minus uncertainty value.

The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

The timing parameters assume a bus timing budget of 20% for output delays and 40% for input setup. This is a reasonable estimate assuming that the process technology is such that APB Bridge can meet this constraint with signals being available after 40% of the clock period.

If a very high frequency bus clock is being used, it is important to ensure that the read data output timing is adequate to meet the requirements of the APB Bridge. In some designs the read data may not be sampled by the APB Bridge, but instead passed directly to the system bus. In this case, it is necessary to ensure that the timing requirements are met for the block that is sampling the data.

The value of 40% of clock period allowed for input setup of APB inputs applies for a style of APB Bridge design with registered inputs that implements logic after the registers. If the APB Bridge has registered outputs then the input setup can be increased, which should result in a reduction of area required.

- c) Timing constraints for non-AMBA SCI-specific signals are set via the `Sci.scr` file. The SCICLK frequency is set to be same as that of PCLK for synthesis. Sample input and output delay constraints are also specified for the other signals. It should be noted that for asynchronous signals, the timing budgets are arbitrary since they can never be accurate. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

5.2 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees

are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

6 INTEGRATION TESTING

The SCI integration test vector suite allows the integrator to verify that the SCI has been connected into the target system correctly. These vectors test 3 classes of signals:

6.1 Integration Test strategy

6.1.1 AMBA APB signals

The APB port signals of the SCI are tested with AMBA APB write and read transactions.

6.1.2 Non-Amba Intra-Chip signals

The SCI intra-chip signals that are connected to the DMA and Interrupt Controllers are testing using the hardware integration test logic incorporated within the periphery of the SCI.

This logic allows values to be written to the SCI intra-chip interrupt and DMA request outputs. The interrupt and DMA controllers also contain integration logic and they read the respective values present on their pins.

Values written to DMA controller clear outputs are read on the DMA clear inputs of the SCI.

6.1.3 Primary I/O Pad signals

The primary I/O Pad signals are directly accessible by a tester.

In the case of primary outputs, values are written to the integration test registers and relayed to the output pin and checked by the tester.

For inputs, values driven by the tester onto the pins are read by performing a read of the register that these are connected to.

6.1.4 Integration Test Code

The integration vectors are contained in the `Sci_Integ.c` file in the `integration/bustest` directory. This file is called by the `Sci.c` file from within in the same directory. As delivered, these vectors are intended to run correctly with the SCI in stand-alone mode i.e. even before the SCI has been integrated into the system. After integration, the following points need to be noted:

- a) The integration vectors targeting the SCI to APB (AMBA signals) connectivity do not have to be modified by the integrator.
- b) The integration vectors targeting the SCI to DMA and SCI to Interrupt Controller (Intra-Chip signals) will need modification. The `Sci_Integ.c` file contains notes listing the changes that the integrator would have to make in the test code.
- c) The integration vectors targeting the SCI to Pad connectivity will have to be modified by the integrator in the absence of any logic that replicates the SCI Trickbox.

7 TEST METHODOLOGIES

The SCI has been designed such that it can be tested using scan-insertion and ATPG.

7.1.1 Scan Insertion and ATPG

For some designs, it will be preferable to use a scan Insertion and ATPG test methodology. It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The SCI design fully supports this scan test and ATPG methodology. Depending on the clocking arrangement used (see **section 4.6**), it will be necessary to use either one or two scan chains. Only the rising edges of clocks are used in the design.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flips-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The main advantage of the scan Insertion and ATPG approach is in cases where significant modifications have been made to the SCI block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that a very high fault coverage can be achieved (99%).

8 FUNCTIONAL AND TIMING VERIFICATION

It is essential to verify the functionality and the timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC

8.1 Functional verification

Functional verification is used to test the functionality of the design. The netlist should also be checked for functional equivalence to the RTL at all the key stages of the design flow. The functional verification can be performed by one of the two methods specified below:

8.1.1 Formal Verification

Formal verification is used to compare RTL with RTL and RTL with netlist. The functionality of the netlist or RTL can be compared to that of the RTL by use of a logic equivalence-checking tool. (For example, Synopsys Formality or Chrysalis). Functionality of the netlist and RTL can be compared within a shorter period of time by this method when compared to gate-level simulations.

8.1.2 Dynamic simulation

Verification tests are run on the RTL and netlist to confirm functionality of the design. Gate-level simulations can be used to compare the netlist functionality to that of the RTL. The same test vectors that were used to verify the RTL are used to verify the netlist functionality. This is however a time consuming process.

8.2 Timing verification

It is essential to verify timing performance of the netlist against the RTL at all the key stages of the design flow. Use of timing verification tools is a useful check for human error in the manual definition of false paths. The following two methods can be used to perform timing verification:

8.2.1 Static Timing analysis

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis.

8.2.2 Dynamic simulation

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results tend to be optimistic since worst case timing paths may not be exercised.

9 CUSTOMISATION

The ARM PrimeCell SCI can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Information to support such modification is included in the SCI (PL131) Design Manual [Ref.3].

9.1.1 FIFO Sizes

The FIFO sizes are defined in the HDL source code to be 8 locations deep, for both receive and transmit logic. These sizes should provide reasonable interrupt latency, adequate for most systems. If different size FIFO buffers are required, for example, to increase allowable latency before overflow or underflow occurs, the FIFO register file and control logic blocks (SciTxRegFile, SciRxRegFile, SciRxFCntl and SciTxFCntl) can be modified accordingly. If this is done, the functional test program and production test vector source must also be modified.

The estimated time to implement these changes is in the region of one person week.

9.1.2 FIFO Fill levels

The FIFO interrupt generation thresholds are set at the programmable tide mark for both the Receive and Transmit FIFOs. This allows the best compromise between good interrupt latency and efficient burst transfer of data across the bus. The fill levels can be modified in the FIFO control logic (SciRxFCntl and SciTxFCntl). If this is done, the functional test program and the production test vector source must be modified accordingly.

The estimated time to implement these changes is in the region of half a person week.

10 APPENDIX A

10.1 PrimeCell Environment Variables

Simulation and Synthesis in the Sci world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the '**GLOBAL**' variable.

e.g. : **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the '**PERIPH**' variable, which should be created as an empty variable.

e.g., **setenv PERIPH <enter>**

The variable is then set to '**Sci**' in the SCI makefile, this enables the make utility to change the variable value for each peripheral in the design.

SIMULATOR

The simulator type is defined by the '**SIMULATOR**' variable.

It can take the following two values

- modelsim - for ModelSim simulations
- LDV - for LDV simulations

e.g. : **setenv SIMULATOR modelsim**

setenv SIMULATOR LDV

TEST_METH

The scan methodology adopted during synthesis is defined by the '**TEST_METH**' variable.

It can take the following three values

- noscan - creates a netlist without scan structures
- scanready - creates a netlist with scan F/Fs but without scan routing.
- scaninsert - creates a fully routed scan netlist.

e.g. : **setenv TEST_METH noscan**

setenv TEST_METH scanready

setenv TEST_METH scaninsert

HDL_SOURCE

The source code type is defined by the '**HDL_SOURCE**' variable.

It can take the following two values

- vhdl [VHDL RTL]
- verilog [Verilog RTL]

e.g. : **setenv HDL_SOURCE vhdl**
setenv HDL_SOURCE verilog

HDL_NETL

The netlist type is defined by the '**HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]
- verilog [Verilog netlist]

e.g. : **setenv HDL_NETL vhdl**
setenv HDL_NETL verilog

TEST_ENV

The test environment is defined by the '**TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

e.g. : **setenv TEST_ENV BUSTALK**
setenv TEST_ENV TICTALK

LIB_VHDL

This location of the vhdl cell library views is defined by the '**LIB_VHDL**' variable.

e.g. : **setenv LIB_VHDL /AVANT/1999.1/cb25/v2.1/vital/cb25os163/zero**

DIRS_VERILOG

This location of the verilog cell library views is defined by the '**DIRS_VERILOG**' variable.

e.g. : **setenv DIRS_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero**

FILES_VERILOG

This location of the verilog UDP file is defined by the '**FILES_VERILOG**' variable.

e.g. : **setenv FILES_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v**

LIB_SYNOP

This location of the synopsys target library files is defined by the '**LIB_SYNOP**' variable.

e.g.: **setenv LIB_SYNOP /AVANT/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models**